

Contexto Técnico — Sprint 5 + Mejoras Skill pr-review

Estado: Pendiente de aprobación **Fecha:** 2026-07-13 **Proyecto:** Gestor-Horarios **Stack:** C++17, Qt6, CMake, OR-Tools (CP-SAT), SQLite

Tabla de Contenidos

1. [Resumen Ejecutivo](#)
 2. [Skill pr-review: 12 Mejoras](#)
 3. [Dependencias Within-Sprint](#)
 4. [Issue #32: Restricciones CP-SAT](#)
 5. [Issue #34: Exportar solución solver](#)
 6. [Issue #35: Exportar horarios manuales](#)
 7. [Issue #31: Validador de datos](#)
 8. [Issue #29: Refinamiento formularios](#)
 9. [Issue #27: Formularios aula + materia](#)
 10. [Matriz de Riesgos](#)
 11. [Orden de Ejecución](#)
 12. [Preguntas Abiertas](#)
-

1. Resumen Ejecutivo

Cambio: sprint5-prreview-improvements

Intención: Preparar el proyecto para Sprint 5 con dos mejoras paralelas:

1. **Skill pr-review:** 12 items de checklist faltantes para cubrir frontend, middleware y backend de forma específica
2. **Contexto técnico Sprint 5:** Detalle de implementación para 6 issues (archivos, clases, flujo de datos, dependencias)

Alcance: Solo planificación y documentación. No se implementa código en este cambio.

Resultado esperado: Archivo MD aprobado por el usuario que sirva como guía de implementación.

2. Skill pr-review: 12 Mejoras

2.1. Decisión de Arquitectura

Opción elegida: Insertar 3 sub-secciones nuevas (5.8 Frontend, 5.9 Middleware, 5.10 Backend) después de la sección 5 existente.

Alternativas descartadas:

- Agregar items inline en tablas existentes → Rechazado: mezclar concerns cross-cutting con module-specific confunde al revisor

- Archivo separado → Rechazado: fragmenta el proceso de revisión

Razón: Las 7 tablas existentes (B, C, A, T, S, E, G) son concerns cross-cutting. Los 12 nuevos items son module-specific. Agrupar por módulo permite escaneo rápido: "estoy revisando un PR de frontend → ver sección 5.8".

2.2. Sección 5.8: Frontend Checklist (5 items)

#	Item	Severidad	Qué buscar
F1	<code>tr()</code> i18n check	HIGH	Strings de UI hardcodeadas sin <code>tr()</code> — botones, labels, mensajes, placeholders
F2	AUTOUI/AUTORCC	HIGH	<code>#include "archivo.ui"</code> sin <code>CMAKE_AUTOUI ON</code> , o <code>.qrc</code> sin <code>CMAKE_AUTORCC ON</code>
F3	Form validation	MEDIUM	QDialog con campos sin validadores, sin feedback de error al usuario
F4	Signal/slot patterns	MEDIUM	<code>SLOT()</code> macro legacy en vez de <code>&Class::method</code> pointer-to-member syntax
F5	Widget file registration	HIGH	Archivo <code>.cpp/.hpp</code> nuevo en <code>src/frontend/</code> sin agregar a <code>FRONTEND_SOURCES</code> o <code>FRONTEND_HEADERS</code> en <code>CMakeLists.txt</code>

2.3. Sección 5.9: Middleware Checklist (2 items)

#	Item	Severidad	Qué buscar
M1	IPC op registration	HIGH	Operación nueva sin agregar constante en <code>messages.h</code> (<code>OP_*</code> pattern)
M2	IPC request validation	MEDIUM	<code>onReadyRead()</code> sin validación de campos requeridos antes de procesar

2.4. Sección 5.10: Backend Checklist (5 items)

#	Item	Severidad	Qué buscar
K1	CP-SAT model validation	HIGH	Variable/constraint sin verificar status del modelo, sin manejo de <code>UNKNOWN/INFEASIBLE</code>
K2	Export format BOM	MEDIUM	CSV export sin BOM UTF-8 (<code>\xEF\xBB\xBF</code>) para compatibilidad Excel
K3	Migration patterns	HIGH	Tabla nueva sin <code>fecha_creacion/fecha_modificacion</code> , sin CHECK constraints, sin migración en <code>migracion.cpp</code>
K4	Template method services	MEDIUM	Servicio nuevo que no sigue patrón <code>Resultado<T></code> — usando <code>bool/QString</code> como retorno

#	Item	Severidad	Qué buscar
K5	Data struct serialization	MEDIUM	Struct nuevo sin <code>toJson()/fromJson()</code> , o sin round-trip test

2.5. Impacto en Tamaño

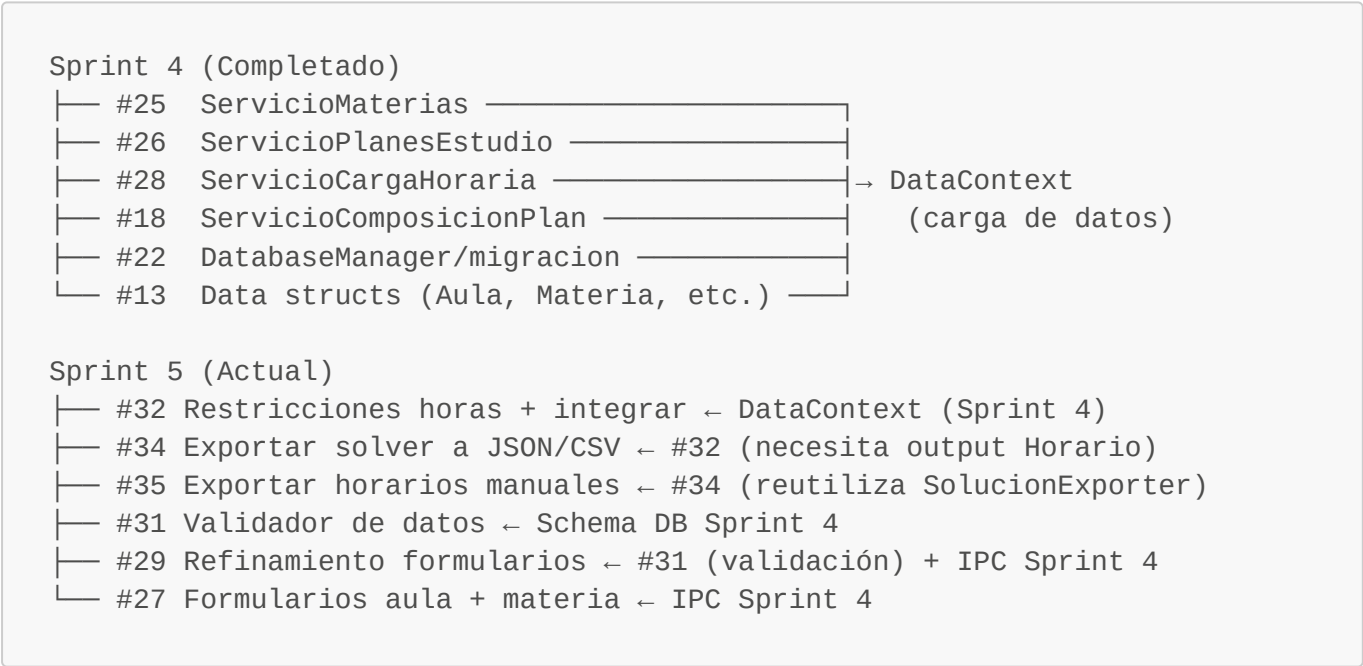
- **Actual:** 706 líneas
- **Con mejoras:** ~946 líneas (~240 líneas nuevas)
- **Acceptable:** Sí — la skill es para consumo de agentes, no para browseo humano

2.6. Archivo a Modificar

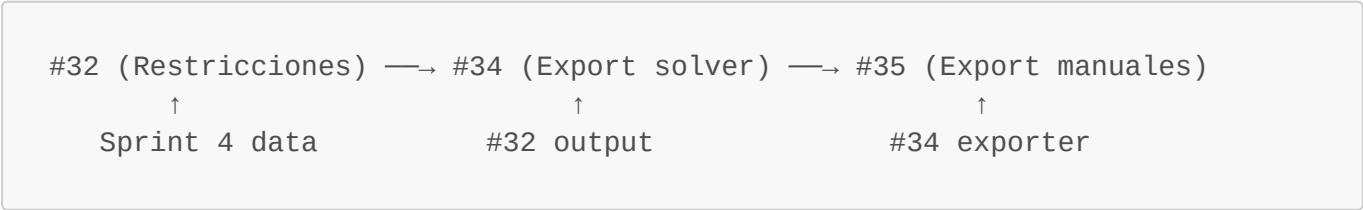
Archivo	Acción	Descripción
<code>.agents/skills/pr-review/SKILL.md</code>	Modificar	Insertar secciones 5.8, 5.9, 5.10 después de línea 256, actualizar sección 1

3. Dependencias Within-Sprint

Grafo de Dependencias



Dependencias Críticas Within-Sprint



#34 depende de #32: No puede empezar sin la struct `Horario` que produce `SolverEngine::solve()`.

Fases de Ejecución Paralela

Fase	Issues	Dependencias
Fase 1 (paralela)	#32, #31, #27	Sin mutuas dependencias
Fase 2 (después de Fase 1)	#34, #29	#34 espera #32, #29 espera #31
Fase 3 (después de #34)	#35	Espera #34

Ruta crítica: #32 → #34 → #35 (3 issues, secuenciales)

4. Issue #32: Restricciones CP-SAT

4.1. Objetivo

Implementar restricción de horas semanales por materia y un orquestador para construir y resolver el modelo de optimización CP-SAT.

4.2. Archivos a Crear

Archivo	Descripción
<code>src/backend/include/backend/solver/IConstraint.hpp</code>	Interfaz abstracta para todas las restricciones
<code>src/backend/include/backend/solver/HorasSemanalesConstraint.hpp</code>	Restricción de horas semanales
<code>src/backend/src/solver/HorasSemanalesConstraint.cpp</code>	Implementación CP-SAT
<code>src/backend/include/backend/solver/RestriccionManager.hpp</code>	Agregador + factory de restricciones
<code>src/backend/src/solver/RestriccionManager.cpp</code>	Aplica todas las restricciones al modelo
<code>src/backend/include/backend/solver/SolverEngine.hpp</code>	Orquestador de creación + resolución del modelo
<code>src/backend/src/solver/SolverEngine.cpp</code>	Ensamblaje del modelo CP-SAT + solve loop
<code>test/test_solver_constraints.cpp</code>	Tests QTest para lógica de restricciones

4.3. Archivos a Modificar

Archivo	Cambio
src/backend/CMakeLists.txt	Agregar nuevos archivos solver a <code>target_sources</code>
test/CMakeLists.txt	Registrar <code>test_solver_constraints</code>

4.4. Diseño de Clases

```
// IConstraint.hpp – Interfaz abstracta de restricción
class IConstraint {
public:
    virtual ~IConstraint() = default;
    virtual void addConstraint(
        const DataContext& data,
        ortools::sat::CpModelBuilder& model,
        const SolverVariables& vars) = 0;
};

// HorasSemanalesConstraint.hpp
// Asegura que cada profesor no exceda el límite semanal de horas
class HorasSemanalesConstraint : public IConstraint {
public:
    explicit HorasSemanalesConstraint(int maxHorasSemanales = 40);
    void addConstraint(const DataContext& data,
        ortools::sat::CpModelBuilder& model,
        const SolverVariables& vars) override;

private:
    int m_maxHoras;
};

// RestriccionManager.hpp
// Factory + agregador de todas las restricciones
class RestriccionManager {
public:
    void addConstraint(std::unique_ptr<IConstraint> constraint);
    void applyAll(const DataContext& data,
        ortools::sat::CpModelBuilder& model,
        const SolverVariables& vars);

private:
    std::vector<std::unique_ptr<IConstraint>> m_constraints;
};

// SolverEngine.hpp
// Orquestador de nivel superior
class SolverEngine {
public:
    explicit SolverEngine(std::chrono::seconds timeout = 30);
    Resultado<Horario> solve(const DataContext& data);

private:
    RestriccionManager m_constraints;
};
```

```
std::chrono::seconds m_timeout;
};
```

4.5. Structs de Datos

```
// DataContext – datos cargados de la DB para el solver
struct DataContext {
    QVector<Aula> aulas;
    QVector<Materia> materias;
    QVector<Profesor> profesores;
    QVector<FranjaHoraria> franjasDisponibles;
    QVector<MateriaEnPlan> planEstudio;
};

// SolverVariables – variables internas del modelo CP-SAT
struct SolverVariables {
    // assignment[a][p][f] = BoolVar: "profesor p imparte materia en aula a
    en franja f"
    std::map<std::tuple<int,int,int>, ortools::sat::BoolVar> assignment;
    // totalHours[p] = IntVar: horas totales asignadas al profesor p
    std::map<int, ortools::sat::IntVar> totalHours;
};
```

4.6. Flujo de Datos

```
DatabaseManager → QSqlQuery → Data structs (Aula, Materia, Profesor,
FranjaHoraria)
↓
DataContext (agrega todos los datos para el solver)
↓
SolverEngine::solve(DataContext)
↓
CpModelBuilder → variables (BoolVar/IntVar por asignación)
↓
RestriccionManager::applyAll() → IConstraint::addConstraint() × N
↓
CpSolver::Solve() → Solution
↓
Horario (QVector<Asignacion>)
```

4.7. Por Qué Estos Patrones

- **IConstraint interface**: Principio Abierto/Cerrado — agregar restricciones sin modificar SolverEngine
- **RestriccionManager como factory**: Centraliza registro. Fácil habilitar/deshabilitar para testing
- **DataContext struct**: Desacopla carga de datos (services) de lógica del solver. SolverEngine nunca toca QSqlDatabase
- **Resultado<Horario>**: Sigue patrón existente de manejo de errores

4.8. Riesgo

Riesgo	Probabilidad	Impacto	Mitigación
Complejidad CP-SAT alta	Media	Alto (bloquea #34, #35)	Empezar primero. Definir DataContext + SolverVariables temprano. Usar skill ortools para ejemplos.

5. Issue #34: Exportar solución solver

5.1. Objetivo

Exportar el **Horario** (output del solver) a formato JSON y CSV.

5.2. Archivos a Crear

Archivo	Descripción
<code>src/backend/include/backend/solver/SolucionExporter.hpp</code>	Métodos estáticos de exportación
<code>src/backend/src/solver/SolucionExporter.cpp</code>	Serialización JSON/CSV
<code>test/test_solucion_exporter.cpp</code>	Tests de round-trip + formato

5.3. Archivos a Modificar

Archivo	Cambio
<code>src/backend/CMakeLists.txt</code>	Agregar <code>SolucionExporter</code> a <code>target_sources</code>
<code>test/CMakeLists.txt</code>	Registrar <code>test_solucion_exporter</code>

5.4. Diseño de Clases

```
class SolucionExporter {
public:
    /** @brief Exporta horario a JSON string. */
    static QString toJson(const Horario& horario);

    /** @brief Exporta horario a CSV con BOM UTF-8. */
    static QString toCsv(const Horario& horario);

    /** @brief Guarda JSON a archivo. */
    static Resultado<bool> guardarJson(const Horario& horario,
                                       const QString& ruta);

    /** @brief Guarda CSV a archivo. */
    static Resultado<bool> guardarCsv(const Horario& horario,
```

```
};                                const QString& ruta);
```

5.5. Formatos de Salida

JSON:

```
{
  "horario": {
    "totalAsignaciones": 45,
    "asignaciones": [
      {
        "materia": "Matematicas",
        "profesor": "Juan Perez",
        "aula": "A101",
        "dia": 1,
        "horaInicio": "08:00",
        "horaFin": "09:30"
      }
    ]
  }
}
```

CSV:

```
Materia,Profesor,Aula,Dia,Hora Inicio,Hora Fin
Matematicas,Juan Perez,A101,Lunes,08:00,09:30
```

BOM prefix: `\xEF\xBB\xBF` para compatibilidad Excel.

5.6. Por Qué Reutilizar `toJson()/fromJson()`

Las data structs (`Aula`, `Materia`, `Profesor`, `Asignacion`, `Horario`) ya tienen métodos `toJson()/fromJson()`. `SolucionExporter` delega a estos métodos, evitando lógica de serialización duplicada. El exporter agrega la capa de formato (headers CSV, BOM, file I/O).

5.7. Dependencia con #32

`SolucionExporter` toma `Horario` como input — esta struct es el output de `SolverEngine::solve()` de #32. El exporter puede desarrollarse en paralelo si la definición de `Horario` se finaliza primero (ya existe en `horario.hpp`).

6. Issue #35: Exportar horarios manuales

6.1. Objetivo

Exportar horarios creados manualmente (almacenados en DB) a JSON/CSV.

6.2. Archivos a Crear

Archivo	Descripción
<code>src/backend/include/backend/services/ServicioExportHorarios.hpp</code>	Servicio que envuelve exporter + DB
<code>src/backend/src/services/ServicioExportHorarios.cpp</code>	Carga desde DB, delega a SolucionExporter
<code>test/test_servicio_export_horarios.cpp</code>	Tests del servicio con QTemporaryDir

6.3. Archivos a Modificar

Archivo	Cambio
<code>src/backend/CMakeLists.txt</code>	Agregar a <code>target_sources</code>
<code>test/CMakeLists.txt</code>	Registrar test

6.4. Diseño de Clases

```
class ServicioExportHorarios {
public:
    explicit ServicioExportHorarios(QSqlDatabase& db);

    /** @brief Exporta un horario almacenado a JSON. */
    Resultado<QString> exportarJson(int idHorario);

    /** @brief Exporta un horario almacenado a CSV. */
    Resultado<QString> exportarCsv(int idHorario);

private:
    QSqlDatabase& m_db;

    /** @brief Carga Horario desde la DB. */
    Resultado<Horario> cargarHorario(int idHorario) const;
};
```

6.5. Flujo de Datos

```
Frontend (GET /schedules/{id}/export?format=json)
↓ IPC
```

```

Middleware::onReadyRead() → OP_EXPORTAR_HORARIO
↓
ServicioExportHorarios::exportarJson(id)
↓
DatabaseManager → QSqlQuery → Horario struct
↓
SolucionExporter::toJson(horario) → QString
↓
Response IPC → Frontend

```

6.6. Por Qué Separado de #34

SolucionExporter es transformación pura de datos (sin dependencia DB). **ServicioExportHorarios** agrega la capa de servicio (acceso a DB, manejo de errores, integración IPC). Separación mantiene cada clase testeable en aislamiento.

7. Issue #31: Validador de datos

7.1. Objetivo

Validar datos de entrada antes de que lleguen a los servicios del backend.

7.2. Archivos a Crear

Archivo	Descripción
<code>src/middleware/include/middleware/validation/DataValidator.hpp</code>	Interfaz de validación
<code>src/middleware/src/validation/DataValidator.cpp</code>	Implementaciones de validación
<code>test/test_data_validator.cpp</code>	Tests de validación

7.3. Archivos a Modificar

Archivo	Cambio
<code>src/middleware/CMakeLists.txt</code>	Agregar archivos de validación a <code>target_sources</code>
<code>test/CMakeLists.txt</code>	Registrar test

7.4. Diseño de Clases

```

struct ValidationError {
    QString campo;
    QString mensaje;
    int codigo;
};

```

```
class DataValidator {
public:
    /** @brief Valida datos de un aula. */
    Resultado<bool> validateAula(const QJsonObject& data);

    /** @brief Valida datos de una materia. */
    Resultado<bool> validateMateria(const QJsonObject& data);

    /** @brief Valida datos de un profesor. */
    Resultado<bool> validateProfesor(const QJsonObject& data);

    /** @brief Valida datos de un plan de estudio. */
    Resultado<bool> validatePlanEstudio(const QJsonObject& data);

private:
    QVector<ValidationError> m_errors;

    void addError(const QString& campo, const QString& msg, int codigo =
-1);
    void clearErrors();
};
```

7.5. Reglas de Validación (desde schema DB)

Entidad	Campo	Regla
Aula	nombre	Requerido, no vacío
Aula	capacidad	Requerido, > 0
Materia	nombre	Requerido, no vacío
Profesor	nombre	Requerido, único
Profesor	email	Requerido, formato válido
PlanEstudio	código	Requerido, no vacío
MateriaEnPlan	curso	Requerido, > 0
MateriaEnPlan	horas	Requerido, > 0

7.6. Por Qué Validación en Middleware

El middleware está entre frontend y backend. Validar aquí significa:

1. Frontend obtiene feedback rápido (sin round-trip al backend)
2. Servicios del backend se enfocan en lógica de negocio
3. Sigue la arquitectura: middleware maneja "transformación y validación de datos"

7.7. Por Qué QTest para Tests

`DataValidator` usa `QJsonObject` y `Resultado<T>` (tipos Qt). Tests necesitan Qt event loop y moc para testing de signals. Sigue convención del proyecto: `add_qtest()`.

8. Issue #29: Refinamiento formularios

8.1. Objetivo

Conectar formularios existentes a IPC, agregar feedback de validación, mejorar UX.

8.2. Archivos a Modificar

Archivo	Cambio
<code>src/frontend/src/forms/teacher_form_dialog.cpp</code>	Agregar conexión IPC, reemplazar stub con <code>InternalClient</code> real
<code>src/frontend/src/forms/teacher_form_dialog.hpp</code>	Agregar miembro <code>InternalClient</code> , spinner, signals de feedback
<code>src/frontend/src/views/teacher_list_widget.cpp</code>	Conectar respuestas IPC, refrescar tabla desde backend
<code>src/frontend/src/views/teacher_list_widget.hpp</code>	Agregar miembro <code>InternalClient</code>

8.3. Diseño de Enfoque

Patrón de conexión IPC (sigue patrón `InternalClient` existente):

```
// En TeacherFormDialog – reemplazar stub
InternalClient* m_client = new InternalClient(this);

connect(m_client, &InternalClient::respuestaRecibida,
        this, &TeacherFormDialog::onRespuestaServidor);

void TeacherFormDialog::guardarProfesor() {
    QJsonObject payload;
    payload["nombre"] = campoNombre->text();
    payload["correo"] = campoEmail->text();
    payload["telefono"] = campoTelefono->text();
    m_client->enviarSolicitud(Middleware::OP_CREAR_PROFESOR, payload);
}

void TeacherFormDialog::onRespuestaServidor(const QJsonObject& respuesta) {
    if (respuesta["code"].toInt() == Middleware::RESP_EXITO) {
        emit profesorGuardado(...);
        accept();
    } else {
        mostrarError(respuesta["data"].toString());
    }
}
```

Mejoras UX:

- Agregar `QLabel` para mensajes de error de validación (texto rojo debajo de campos)
- Agregar `QProgressBar` o estado deshabilitado durante llamada IPC
- Atajo de teclado: Enter para guardar, Escape para cancelar
- Auto-focus en primer campo al abrir

8.4. Por Qué Modificar Archivos Existentes

Issue #29 mejora formularios YA EXISTENTES (#27 crea formularios NUEVOS para aula/materia). Este issue conecta `TeacherFormDialog` existente a IPC real y agrega polish de UX. No hay nuevas clases — solo mejorar las existentes.

9. Issue #27: Formularios aula + materia

9.1. Objetivo

Crear nuevos diálogos de formulario y widgets de lista para Aulas y Materias.

9.2. Archivos a Crear

Archivo	Descripción
<code>src/frontend/src/dialogs/AulaFormDialog.hpp</code>	Formulario de aula (QDialog)
<code>src/frontend/src/dialogs/AulaFormDialog.cpp</code>	Implementación
<code>src/frontend/src/dialogs/MateriaFormDialog.hpp</code>	Formulario de materia (QDialog)
<code>src/frontend/src/dialogs/MateriaFormDialog.cpp</code>	Implementación
<code>src/frontend/src/widgets/AulaListWidget.hpp</code>	Lista de aulas con tabla
<code>src/frontend/src/widgets/AulaListWidget.cpp</code>	Implementación
<code>src/frontend/src/widgets/MateriaListWidget.hpp</code>	Lista de materias con tabla
<code>src/frontend/src/widgets/MateriaListWidget.cpp</code>	Implementación

9.3. Archivos a Modificar

Archivo	Cambio
<code>src/frontend/CMakeLists.txt</code>	Agregar nuevos archivos a <code>FRONTEND_SOURCES/FRONTEND_HEADERS</code>
<code>src/frontend/src/views/main_window.cpp</code>	Conectar nuevos widgets en <code>QStackedWidget</code>

9.4. Diseño de Clases

```
// AulaFormDialog – sigue patrón TeacherFormDialog
namespace gestor::frontend::dialogs {
class AulaFormDialog : public QDialog {
    Q_OBJECT
```

```

public:
    explicit AulaFormDialog(QWidget* parent = nullptr);
    void cargarDatos(const QString& nombre, int capacidad,
                    const QString& edificio, const QString& piso);
signals:
    void aulaGuardada(const QString& nombre, int capacidad,
                     const QString& edificio, const QString& piso);
private slots:
    void guardarAula();
private:
    QLineEdit* campoNombre;
    QSpinBox* campoCapacidad; // <-- usa QSpinBox para validación numérica
    QLineEdit* campoEdificio;
    QLineEdit* campoPiso;
    QLabel* etiquetaError;    // <-- feedback de validación
    QPushButton* botonGuardar;
    QPushButton* botonCancelar;
};
}

// AulaListWidget – sigue patrón TeacherListWidget
class AulaListWidget : public QWidget {
    Q_OBJECT
public:
    explicit AulaListWidget(QWidget* parent = nullptr);
private slots:
    void abrirFormularioNueva();
    void agregarAulaATable(const QString& nombre, int capacidad,
                          const QString& edificio, const QString& piso);
private:
    QTableWidget* m_table;
    QPushButton* m_registerButton;
    InternalClient* m_client; // conexión IPC
};

```

9.5. Integración CMakeLists

El CMakeLists del frontend usa listas planas `FRONTEND_SOURCES`/`FRONTEND_HEADERS` (no `target_sources()`). Nuevos archivos deben agregarse a ambas listas:

```

set(FRONTEND_SOURCES
    # ... existente ...
    src/dialogs/AulaFormDialog.cpp
    src/dialogs/MateriaFormDialog.cpp
    src/widgets/AulaListWidget.cpp
    src/widgets/MateriaListWidget.cpp
)

set(FRONTEND_HEADERS
    # ... existente ...
    src/dialogs/AulaFormDialog.hpp

```

```
src/dialogs/MateriaFormDialog.hpp
src/widgets/AulaListWidget.hpp
src/widgets/MateriaListWidget.hpp
)
```

Nota: Headers están en `src/` (no `include/frontend/`) siguiendo el patrón existente donde `teacher_form_dialog.hpp` y `teacher_list_widget.hpp` viven junto a sus archivos `.cpp`.

9.6. Integración en MainWindow

`MainWindow::setupCentralArea()` usa `QStackedWidget`. Nuevos widgets se agregan como páginas:

```
// En main_window.cpp setupCentralArea():
m_aulaListWidget = new AulaListWidget(this);
m_contenedorVistas->addWidget(m_aulaListWidget); // index 2

m_materiaListWidget = new MateriaListWidget(this);
m_contenedorVistas->addWidget(m_materiaListWidget); // index 3
```

10. Matriz de Riesgos

Issue	Riesgo Técnico	Complejidad	Dependencias Bloqueantes	Mitigación
#32 Restricciones	ALTO	ALTO	Data structs Sprint 4	Empezar primero. Definir DataContext + SolverVariables temprano. Usar skill ortools para ejemplos.
#34 Export solver	MEDIO	MEDIO	#32 (output Horario)	Puede empezar lógica de export en paralelo si Horario struct es estable (ya existe en horario.hpp).
#35 Export manuales	BAJO	BAJO	#34 (SolucionExporter)	Wrapper delgado sobre #34. Riesgo bajo si #34 es sólido.
#31 Validador	BAJO	MEDIO	Schema DB Sprint 4	Schema estable. Reglas bien definidas.
#29 Refinamiento	MEDIO	MEDIO	#31 (validación)	Riesgo: conexión IPC puede exponer race conditions. Mitigación: reusar patrón InternalClient existente.
#27 Formularios	BAJO	MEDIO	IPC Sprint 4	Sigue patrón TeacherFormDialog exactamente. Riesgo bajo — mostly boilerplate.

Resumen de Riesgos

Riesgo	Probabilidad	Impacto	Mitigación
#32 CP-SAT modelo muy complejo	Media	Alto (bloquea #34, #35)	Empezar con restricciones mínimas. Usar skill ortools. Definir interfaz SolverVariables temprano.
Skill pr-review sobrecargada	Baja	Bajo	3 sub-secciones enfocadas, no items dispersos. ~240 líneas agregadas — manejable.
Frontend CMakeLists estructura plana	Baja	Medio	Seguir patrón existente exacto: agregar a FRONTEND_SOURCES/FRONTEND_HEADERS.
Validación middleware dependencia circular	Baja	Alto	DataValidator en middleware Qt-only (JsonObject). Sin dependencia backend.

11. Orden de Ejecución

Fase 1 (Paralela — sin dependencias mutuas)

Issue	Asignado	Archivos	Esfuerzo Estimado
#32 Restricciones	Luis	7 crear + 2 modificar	ALTO
#31 Validador	Manuel	3 crear + 2 modificar	MEDIO
#27 Formularios	Paola	8 crear + 2 modificar	MEDIO

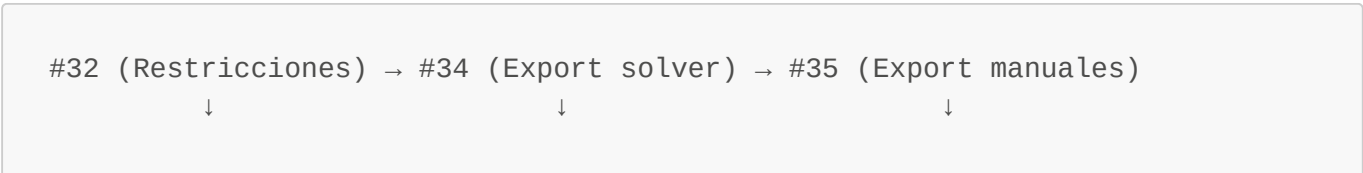
Fase 2 (Después de Fase 1)

Issue	Asignado	Espera	Archivos	Esfuerzo Estimado
#34 Export solver	Luis	#32	3 crear + 2 modificar	MEDIO
#29 Refinamiento	Dani	#31	4 modificar	MEDIO

Fase 3 (Después de #34)

Issue	Asignado	Espera	Archivos	Esfuerzo Estimado
#35 Export manuales	Nicole	#34	3 crear + 2 modificar	BAJO

Ruta Crítica



Fase 1, Issue 1

Fase 2, Issue 1

Fase 3, Issue 1

Duración mínima: 3 fases secuenciales. Si #32 se retrasa, #34 y #35 se retrasan.

12. Preguntas Abiertas

Antes de implementar, necesito clarificación sobre:

1. ¿Existe tabla **Horario** en la DB para horarios manuales? (necesario para #35)
 2. ¿Debería **SolverEngine** soporte resolución incremental (agregar constraints dinámicamente) o solo batch?
 3. ¿Hay timeout máximo para resolución CP-SAT que deba ser configurable vía IPC?
-

Aprobación

Para aprobar este documento, responde con:

- **"Aprobado"** para proceder a implementación
 - **"Modifico X"** para indicar cambios necesarios
 - **"Rechazo"** para detener el proceso
-

Documento generado por SDD Orchestrator — Gestor-Horarios Última actualización: 2026-07-13